\t	horizontal tab (HT or 0x09 (9) in ASCII)
\v	vertical tab (VT or 0x0B (11) in ASCII) (since PHP 5.2.5)
\e	escape (ESC or 0x1B (27) in ASCII) (since PHP 5.4.4)
\f	form feed (FF or 0x0C (12) in ASCII) (since PHP 5.2.5)
\\	backslash
\$	dollar sign
\"	double-quote
\[0-7]{1,3}	the sequence of characters matching the regular expression is a character in octal notation, which silently overflows to fit in a byte (e.g. "\400" === "\000")
\x[0-9A-Fa-f]{1,2}	the sequence of characters matching the regular expression is a character in hexadecimal notation
\u{[0-9A-Fa-f]+}	the sequence of characters matching the regular expression is a Unicode codepoint, which will be output to the string as that codepoint's UTF-8 representation (added in PHP 7.0.0)

As in single quoted strings, escaping any other character will result in the backslash being printed too. Before PHP 5.1.1, the backslash in *\{$var}* had not been printed.

The most important feature of double-quoted strings is .the fact that variable names will be expanded

4.1.3 Heredoc

A third way to delimit strings is the heredoc syntax: <<<. After this operator, an **identifier** is provided, then a newline. The string itself follows, and then the same identifier again to close the quotation.

The closing identifier *must* begin in the first column of the line. Also, the identifier must follow the same naming rules as any other label in PHP: it must contain only alphanumeric characters and underscores, and must start with a non-digit character or underscore.

It is very important to note that the line with the closing identifier must contain no other characters, except a semicolon (;). That means especially that the identifier *may not be indented*,

Keyword

Identifier is a name that identifies (that is, labels the identity of) either a unique object or a unique class of objects, where the "object" or class may be an idea, physical [countable] object (or class thereof), or physical [noncountable] substance (or class thereof).